

Case Study: How Logistics Handle Product Delivery

Route Planning • Delivery Tracking • Traffic-Aware Delivery

1. Problem Statement

In e-commerce logistics, delivering a product is not only about moving a package from one place to another. The logistics system must decide which route a delivery vehicle should take, monitor the delivery, and respond when road or traffic conditions change.

The main problem considered in this case study is how logistics can deliver products efficiently when there are many possible routes and changing real-life conditions such as traffic, road delays, and different delivery locations.

Key Problems

- Finding an efficient route between the current location and the customer's location.
- Handling many possible roads and destinations.
- Choosing a route based on distance or expected travel time.
- Reacting when traffic conditions change during delivery.
- Keeping delivery information updated as the package moves.
- Delivering multiple orders efficiently instead of treating every delivery as an isolated trip.

Why Route Planning Matters

A route that looks shortest on a map may not always be the fastest route. For example, a 5 km road with heavy traffic can take longer than an 8 km road with normal traffic. Therefore, a logistics system should consider more than just distance.

This creates an engineering problem: how can a system choose a good route quickly and then adjust it when the real-world situation changes?

2. Proposed Solution by Engineers/Developers

The proposed solution is a delivery management system that combines route-finding algorithms, delivery tracking, and current traffic information. The system represents roads and locations as a network and uses an algorithm to find a suitable route.

A simplified delivery flow is:

1. Order received
2. Package prepared and assigned to a courier
3. Current location and destination identified
4. Suitable route calculated
5. Courier starts delivery
6. Traffic and delivery events are monitored
7. Route is recalculated if conditions change
8. Package reaches the customer and delivery is completed

Main Idea

The engineers do not need to manually decide every route. Instead, software uses location data, road information, route algorithms, and traffic information to help make the decision.

3. System Architecture

The following is a proposed architecture for the case study. It describes how engineers could connect the major parts of a logistics delivery system. It is a simplified model and is not a claim about Daraz's private internal architecture.

Component	Main Responsibility
Customer / Order System	Provides the delivery order and destination information.
Delivery Management System	Manages packages, couriers, delivery status, and assignments.
Location / GPS Service	Provides the current location of the delivery vehicle.
Traffic Data Service	Provides current or recent information about road conditions.
Route Engine	Uses a route algorithm to calculate a suitable path.
Tracking Database	Stores order, route, location, and delivery-event information.
Notification Service	Sends important delivery updates to the customer.
Courier App / Tracking UI	Shows the route, delivery information, and current status.

Simple System Flow

Order Information → Delivery Management → Current Location + Traffic Data → Route Engine → Courier App → Tracking Database → Customer Updates

During delivery, the route engine can use the current location and available traffic information to decide whether the existing route is still suitable. If conditions change significantly, the route can be recalculated.

Route-Finding Algorithms

Two important algorithms that can be discussed in this case study are Dijkstra's algorithm and A* (A-Star). They are general path-finding techniques that can be used as part of a route engine.

Dijkstra's Algorithm

Dijkstra's algorithm finds the lowest-cost path through a network when the edge costs are non-negative. For logistics, the cost could represent distance or travel time.

Simple idea: start from the current location, compare the cost of possible paths, keep the lowest known cost, and continue toward the destination.

A* Algorithm

A* also finds paths, but it uses both the cost already travelled and a heuristic estimate of the remaining cost to the destination. This helps it focus its search toward the goal.

Simple idea: A* considers what the route has already cost plus an estimate of how much remains.

Heuristics and Real-Life Traffic Tracking

Heuristics

A heuristic is an estimate used to guide a search algorithm. In A*, a common example is the estimated distance from the current location to the destination. The heuristic helps the algorithm prioritize routes that appear to move toward the destination.

For example, if two possible roads are available and one appears to lead much closer to the destination, A* can give that direction more attention. The heuristic is an estimate, so it does not necessarily equal the actual travel time.

Real-Life Traffic Tracking

Route planning based only on a static map has a limitation: road conditions change. A logistics system can use current traffic information to estimate how long different roads may take.

Possible sources of traffic information include:

- GPS/location information from delivery vehicles.
- Traffic information services.
- Road or traffic sensors where available.
- Recent travel-time information collected by the system.

Suppose Route A is 5 km but has heavy traffic, while Route B is 8 km and has normal traffic. If Route A is expected to take 30 minutes and Route B 15 minutes, the longer route may be the better delivery choice.

How Routing and Traffic Work Together

The route engine can treat travel time as a cost. When traffic increases the expected travel time of a road, that road becomes less attractive. The system can then recalculate the route and select another path if it is expected to be more efficient.

Example Delivery Flow

Step	What happens
1	Courier receives the package and starts from the current location.
2	System gets the destination and available road information.
3	Route engine calculates a suitable route.
4	Courier follows the route using the courier app.
5	Location and traffic information change during the trip.
6	System checks whether the current route is still suitable.
7	If necessary, a new route is calculated.
8	Courier reaches the destination and marks the delivery complete.

4. Technology Stack and Why It Is Selected

The following is a proposed technology stack for implementing the case-study solution. It is presented as an engineering design, not as a statement of Daraz's actual private technology stack.

Technology / Area	Why it can be used
Frontend: React	Builds a clear courier and tracking interface with reusable components.
Backend: Python + Django / REST API	Provides APIs for orders, delivery events, route requests, and tracking data.
Database: PostgreSQL	Stores structured information such as orders, drivers, routes, and delivery events.
Maps / GPS Service	Provides location and map information needed for route planning.
Routing Engine: A* / Dijkstra	Calculates suitable paths through the road network.
Traffic Data API	Provides current or recent traffic information for travel-time decisions.
Real-time Communication: WebSocket	Can push updated delivery information to a tracking interface without requiring constant manual refresh.
Notification Service	Sends important delivery updates to customers or delivery staff.

5. Features Implemented / Proposed

For this case study, the main features are:

- Order and package assignment.
- Courier location tracking.
- Route calculation using path-finding algorithms.
- Traffic-aware route selection.
- Route recalculation when conditions change.
- Delivery status updates such as Dispatched, In Transit, Out for Delivery, and Delivered.
- Tracking history for delivery events.
- Customer notifications for important delivery changes.
- Courier interface for route and delivery information.
- Customer tracking interface showing the latest delivery status.

Important Terminologies

Term	Simple Meaning
Node	A location or point in a road network.
Edge	A road or connection between two nodes.
Path	A sequence of roads from a starting point to a destination.
Cost	A value used to compare paths, such as distance or travel time.
Heuristic	An estimate used by A* to guide the search toward the destination.
Real-time Tracking	Updating location or delivery information as new data becomes available.
Re-routing	Calculating a new route when the current route is no

	longer suitable.
--	------------------

6. Overall Summary — Solving the Logistics Problem

The main challenge in logistics is making delivery efficient while dealing with a changing real-world environment. A route that is good at the beginning of a trip may become less useful because of traffic, road conditions, or other delays.

The proposed solution is to combine a delivery management system with route-finding algorithms, location data, traffic information, and tracking. Dijkstra's algorithm can find a lowest-cost path based on known road costs, while A* can use a heuristic to guide the search toward the destination. Traffic information can change the cost of roads and cause the system to recalculate a route when needed.

End-to-End Solution

Order → Package Assignment → Location Data → Route Calculation → Courier Delivery → Traffic Monitoring → Re-routing if Required → Delivery Status Update → Customer Notification → Completed Delivery

How Engineers Solve the Problem

- First, identify what information is needed: destination, current location, road network, and traffic conditions.
- Represent the road network so a computer can compare different paths.
- Use a path-finding algorithm to calculate a suitable route.
- Use a heuristic with A* when a goal-directed search is useful.
- Update route costs when traffic information changes.
- Recalculate the route when the current route becomes inefficient.
- Store delivery and tracking events so the delivery journey can be monitored.
- Provide clear information to the courier and customer.

Expected Result

The expected result is a logistics system that can select efficient routes, respond to changing traffic conditions, keep delivery information updated, and help couriers complete deliveries more reliably. From the customer's point of view, this also provides clearer delivery progress and more useful status information.

Conclusion

Efficient logistics is a combination of software, data, maps, algorithms, and real-world delivery operations. Dijkstra's algorithm, A*, heuristics, GPS/location tracking, and traffic information are useful concepts for understanding how a modern delivery system can plan and adjust routes. The proposed architecture connects these parts into one simple delivery workflow.

Note on Scope

This case study explains a proposed engineering approach to logistics delivery. Public information does not reveal all of a company's private internal systems, so the architecture and technology stack above should be understood as a simplified design for the case study rather than a claim about any specific company's internal implementation.

Reference

Dijkstra, E. W. (1959).: A Note on Two Problems in Connection with Graphs.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968): A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

Google Maps Help — See Traffic Near You. Useful for understanding real-time traffic information, delays, crashes, construction, and road closures.

OpenStreetMap — Routing and Map Data. Useful for understanding road-network data and routing applications.